

오픈소스 기반 반도체 칩 종합 설계 2주차_[추가]

[우주 침략자 게임 만들기]

권효재

지난 학습 에세이에서 부족하게 작성했던 부분들을 다시 살펴보고, 학습노트와 실제 코드를 보면서 내용을 정리해보았다. 추가 과제도 진행하면서 이전에 이해했던 내용 중 다시 확인할 부분도 같이 살펴보았다.

과제1. Q.1 과 Q.4 의 베릴로그 RTL에서 그래픽 장치를 다루는 방식의 차이점

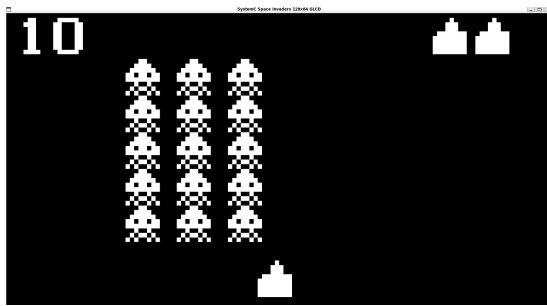
Q1과 Q4의 코드를 비교해보았다. Q1에서는 vga_x, vga_y가 입력으로 들어오기 때문에 외부에서 현재 화면 좌표를 넣어주고, 좌표 자체를 게임 엔진이 만드는데 아니라, 게임 엔진에서는 그 좌표가 어떤 것인지 위치확인 RGB 값만 정해주지만, Q4에서는 lcd_x, lcd_y가 출력으로 나간다. 내부의 scan_x, scan_y를 계속 증가시키면서 128*64 화면 좌표를 직접 만들고, 그 위치에 픽셀이 필요한지 판단해서 lcd_data, lcd_write_en이랑 같이 내보내는 방식이다..

왜 vga_x, vga_y와 lcd_x, lcd_y의 입출력 방향이 왜 반대일까 했었는데, 구조를 이해하니 알 수 있었다.

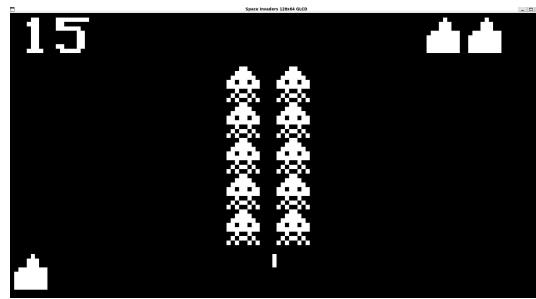
과제2. Q.10 실습 수행. 스프라이트 시현 방식에 대하여 아두이노 C++와 베릴로그 RTL의 차이를 비교 설명 할 것.

아두이노의 C++에서는 drawBitmap() 같은 함수를 이용해서 비트맵을 화면에 그리고 OLED로 전송하지만, verilog에서는 스프라이트 배열의 비트를 현재 scan_x, scan_y 좌표와 직접 비교해서 픽셀을 출력한다. 아두이노는 함수를 실행해서 화면을 그리는 방식이고, verilog는 회로가 현재 좌표에 따라 화소 값을 계속 결정하는 방식이다. Q10 실습에서는 verilog의 스프라이트 출력이 정상적으로 되는지 확인하면서, 좌표 계산이나 비트맵 배열을 처리할때 오류가 생길 수 있는 부분도 같이 확인했다. verilog는 지금 좌표를 기준으로 직접 픽셀을 결정하기 때문에, 좌표 계산이 조금만 잘못돼도 화면에 바로 이상하게 나타날 수 있다는 점을 확인할 수 있었다.

과제3. Q.11 실습 수행. 좌우 화살표 키보드 입력에 대하여 이상 반응을 보이는 이유를 찾을 것. Q.11의 SystemC 테스트 벤치가 변경된 점을 찾아볼 것. 키보드 입력을 보완하기 위해 베릴로그 RTL의 차이점을 찾아볼 것



Q.10 시뮬레이션



Q.11 시뮬레이션

Q11에서는 systemC 테스트벤치에서 SDL을 이용해서 실제 PC 키보드 입력을 받는다. SDL_PumpEvents()와 SDL_GetKeyboardState()로 방향키와 스페이스바 상태를 확인하고,

이 값을 btn_left, btn_right, btn_fire 신호로 rtl 코드로 전달한다. 이전 systemC 테스트벤치와 비교하면 키 입력 방식 자체는 크게 달라지지 않았고, 길게 작성되어 있던 if-else 부분이 삼항 연산자로 정리되면서 코드가 조금 더 간단해졌다. (이부분은 지난주 과제에서 Q11에서 잘못 진행했던 diff 비교로, 뒷부분에 수정 내용을 추가했다.) 좌우 방향키가 너무 빠르게 반응하는 이유는 키 입력 신호가 한 번 1이 되면 다음 키 상태를 확인하기 전까지 계속 유지되고, RTL에서는 그 값을 매 클럭마다 처리하기 때문이다. 테스트벤치는 약 16.6ms마다 키 상태를 확인하지만 RTL은 50MHz로 동작해서, 입력이 유지되는 동안 cannon_x가 여러 번 바뀌어 플레이어가 너무 빠르게 움직인다. 또 Q10에서는 왼쪽 이동, 오른쪽 이동, 발사 조건이 if-else if로 이어져 있었는데 Q11에서는 각각 따로 if문으로 처리하도록 바뀌어 있었다. 그래서 이동하면서 동시에 발사하는 동작도 가능해졌다.

과제4. Q.13의 시뮬레이션 디렉토리에서 실습을 실시할 것



Q.13 시뮬레이션 (확대_128*64 픽셀)

Q13에서는 simulation 디렉토리에서 여러 RTL 파일 중 하나를 선택해서 실행할 수 있었는데, 그중에서 4_space_invaders_engine_glcd.v.Ok 를 선택해서 시뮬레이션해보니 작은 128x64 크기의 GLCD 창이 열렸다. 화면에 표시되는 내용은 verilog RTL 내부에서 scan_x, scan_y를 기준으로 픽셀 값을 만들고, lcd_x, lcd_y, lcd_data 같은 신호를 통해 SystemC 쪽 GLCD 모델로 전달되는 방식이었다. Q13에서는 Verilog 코드만 따로 확인하는 게 아니라 SystemC 테스트벤치와 가상 GLCD까지 같이 연결된 상태에서 동작을 볼 수 있었다. Ok 버

전 외에도 bullet_no_disp, btn_fire, FPGA_err 같이 오류가 있는 파일도 따로 준비되어 있어서, 어떤 코드 문제가 있을 때 시뮬레이션 결과가 어떻게 달라지는지도 직접 비교해볼 수 있었다.

과제+. 지난 과제의 잘못된 점

다른 학우분들의 과제를 보고, 이번 실습과제를 진행해본 결과 지난주 과제에서 diff 비교 대상을 잘못 잡고 진행했다는 것을 알게 되었다. diff 명령을 Q.11_SC_Final 내부에서 진행해서, if-else문이 삼항 연산자로 줄어들었다거나 코드가 더 간단하게 정리되었다는 식으로 작성했다. 현재 경로와 비교 대상 파일을 제대로 확인하지 않고 Q.11_SC_Final의 파일과 비교하다 보니, 과제에서 요구한 비교와는 다른 결과를 분석하게 되었다. 교안을 다시 확인해 파일끼리 비교해보니 주석이나 로그 문구, 코드를 정리한 부분이 대부분이었다. 앞으로는 명령어를 실행하기 전에 어떤 파일을 비교하는지 먼저 확인하고, 학습노트에서 교수님께서 요구한 것이 맞는지 확인한 뒤 진행해야겠다고 생각했다. 낯선 내용들 속에서 내가 뭘 하고 있는지 항상 재인지 해야겠다는 생각이 들었다.

과제5. SystemC에 대한 이해 높이기

verilog로 테스트벤치를 작성할 수 있지만, C++의 하드웨어 병렬 커널이 들어있는 라이브러리인 systemC를 이용해서 추상성이 높은 시스템 수준에 대해서 검증하고 시뮬레이션 하는 것을 가능하게 한다. 설계 규모가 엄청나게 커지고 이에 따라서 높은 추상적인 방법이 모색되고 있는데, 이것 해결해주는 방안이다. systemC 헤더파일에서 가져온 클래스들을 통해 하드웨어 객체 같은 것들을 통해서 클래스 객체로 클럭을 생성시키고 사건함수 구동을 지정한다. 하드웨어의 병렬성을 구현하기 위해서는 verilog 처럼 사건구동과 지연할당이 필요한데, 이런 요건들을 순차성이 기본인 소프트웨어 언어에 클래스 라이브러리와 병렬 시뮬레이션 엔진으로 해결하는 것이다.

과제6. SW언어와 HW언어 비교

하드웨어 언어로 작성했다고 모두 합성 가능한 것은 아니다. verilog로 작성한 테스트벤치 코드의 경우에는 합성이 불가능하다. 따라서 레지스터 간 데이터 이동이 표현된 RTL이어야만 합성이 가능하다. 디지털 회로는 0과 1의 논리값을 가지는 신호들로 구성되기 때문에 하드웨어 언어는 비트 단위로 표현하고 제어할 수 있어야 한다. 소프트웨어 언어인 C++은 이런 하드웨어 신호의 비트 폭을 중심으로 만들어진 언어는 아니기 때문에, SystemC에서는 sc_uint 나 sc_logic 같은 자료형을 추가해서 하드웨어 신호를 비트 단위로 표현할 수 있게 한다. 소프트웨어 언어에서는 논리연산에 0,1 두개의 논리값이 있지만, 하드웨어 언어의 경우 X,Z가 추가되어, 실제 디지털 회로에서 발생할 수 있는 unknown과 high impedance 상태를 표현할 수 있어야 한다. 이것 또한 마찬가지로 하드웨어 설계를 목적으로 다루는 언어이기 때문이다. 하드웨어 언어 중 verilog는 클럭에 의해 동시에 작동해야 하는 이유로 병렬성을 기본으로 하고 절차적 할당문의 always문을 사용하여 순차성을 갖추는데, 이 중에서도 지연할당을

사용해야 병렬성 속에서 edge나 event가 주어졌을 때 신호 감지에 의해서 순차성을 가질 수 있다. 하지만 소프트웨어 언어의 경우 기본적으로 순차성을 가지고 systemC와 같이 내장된 병렬 시뮬레이션 커널을 사용하는 등의 방법을 통해 병렬성을 구현한다. C++의 경우 코드 실행 순서는 코드의 위치, 연산자의 위치 등에 의해서 결정할 수 있는고, 여러 함수 등에 인수를 줘서 시간 관리가 가능하다., C++의 경우 시뮬레이션 커널이 콜백 함수를 호출하는 방식을 사용하는데, 메쏘드나 쓰레드를 프로세스로 등록하고, 신호의 edge나 event가 발생했을 때 시뮬레이션 커널이 해당 프로세스를 실행하도록 스케줄링한다.

이번 과제를 진행하면서 내가 부족하고 중요한 부분들을 다시 확인해볼 수 있었다. SystemC와 Verilog RTL이 각각 어떤 역할을 하고, 키보드 입력이나 그래픽 출력이 서로 어떻게 연결되는지 다시 보는 과정이 도움이 됐다. 또 정리해보는 시간을 가지니 완전히는 아니더라도 어느정도 흐름이 구조화가 된 것 같다.